Further Lean proof simplifications

Order rigidity, residue representatives, source positivity, and integer certificates

Prepared for Vladimir Reshetnikov 26 October 2026

Result and recommendation

This packet proposes six targeted improvements after treating the earlier proof-simplification corpus as an assumed completed baseline. Five are local proof-body changes in five maintained modules. Together they remove **41 physical source lines**, including the new direct mathlib import. The sixth strengthens the assembled-source positivity theorem by using a selected tile digit. Its API migration removes three further lines, giving **44 lines removed in the recommended combination**; a separately costed compatibility alternative is also supplied.

The strongest immediate deletion replaces a custom finite-ordinal induction with mathlib's rigidity theorem for injective monotone endomorphisms of finite linear orders. Three controller edits replace repeated representative calculations with the existing `Nat.ModEq` and `ZMod` APIs. A smaller arithmetic edit proves a nonzero certificate by square positivity and integer discreteness, eliminating the sign split and nonlinear tactics. The positivity improvement removes a logical dependence on a marked tiling at one precise source-construction boundary.

Verification scope. The original and replacement finite-order and integer-certificate proof kernels compile with the repository-pinned Lean 4.32.0 and mathlib revision. Controller proofs compile both independently and in an isolated four-module chain with a documented narrower validation import. The positivity kernel and a source-shaped adapter compile independently. The complete maintained project and its full downstream import closures were not rebuilt. Production patches are therefore reviewable integration candidates, with the checked scope recorded for each.

ID	Exact target	Before	After
P1	`WangTiles942PhysicalOrder.lean`	180	155
P2	`ControllerIntervalSums.lean`	83	80
P3	`ControllerResidueLift.lean`	68	63
P4	`ControllerAuxiliaryChoice.lean`	50	45
P5	`ArithmeticNonzero.lean`	68	65
	Complete-module totals for P1–P5	449	408

These are measured text changes against the pinned checkout. They are not estimates of cumulative savings after arbitrary overlapping implementations of earlier proposals, and they are not performance measurements. With P6's source module and its check client included, the recommended combination changes 609 lines to 565. P6 is accounted for separately in Section 5.

1 Source pins and the assumed prior baseline

The initial source review is pinned to `7df43cdfacb490cd3ca3acd23e39027efebf8801`. A closing refresh examined `663fb267f6142ca4c0ea5bcd2d4f607f9a4b0eda`. The seven target files, including the positivity check client, are byte-identical between these commits. The refresh changed six other maintained Lean modules and the adoption ledger; it added no proposal manuscript, patch, or incoming archive. The exact paths and SHA-256 comparisons are in `evidence/refresh_audit.json`.

The project pins `leanprover/lean4:v4.32.0`. Its mathlib revision is

`81a5d257c8e410db227a6665ed08f64fea08e997`.

All compilation receipts in this packet use those versions. The initial maintained-tree inventory contains 664 Lean files and 84,248 physical lines. The review was focused by proof obligation and candidate, rather than an assertion that every line of that tree received equal scrutiny.

The exclusion corpus comprises the 23 placed packets `2026-10-26a` through `2026-10-26w`, all five ZIP archives still in `docs/incoming`, and the incoming index. The audit inventories 2,329 files and archive-member payloads, with 2,161 searchable text files and 1,042 incoming archive members. Its 175 coverage records overlap: they are not 175 independent proposals. See the placed-packet index and the bundled manifests.

The overlap method includes complete file/member inventories and hashes, all main manuscript section indexes, proposal descriptions and shared exclusion catalogs, relevant technical notes, and candidate-specific full-text searches over prose and proposed patches. Unchanged source snapshots were distinguished from actual proposals. This is not a fresh semantic certification of every old implementation, or a traversal of every historical link in the incoming index.

The user's assumed-adoption instruction is applied to notes-only, deferred, uncompiled and rejected proposals as well as adopted code. Thus scanner iteration/fusion, witness-layout transport, PR composition automation, circuit certificates, coefficient frameworks, carry uniqueness, generic modular projection, Pell sign interfaces, rank cast bridges, and the earlier Wang-row abstractions are all excluded as contributions of this packet. A new client of a promised old abstraction is not counted as a new idea. Each retained item below instead identifies a concrete additional proof obligation or an unnecessarily strong theorem premise.

2 P1: finite-order rigidity replaces strong induction

Target. `WangTiles942PhysicalOrder.strictMono_fin_eq_self`.

The maintained proof establishes that a strictly increasing self-map of `Fin` fixes every element. It first turns injectivity into surjectivity, then performs strong induction on the underlying natural index. At each index it obtains a preimage and compares that preimage with the index, splitting into smaller, equal and larger cases. This is a complete proof of a fact already packaged in the pinned library.

For any finite linearly ordered type A, an injective monotone map $f : A \to A$ is the identity. Mathlib provides this as `OrderHom.eq_id_of_injective`. Strict monotonicity supplies both required facts. Equality of the bundled maps can then be evaluated at the desired element:

```
theorem strictMono_fin_eq_self {n : ℕ} (f : Fin n → Fin n)
    (hf : StrictMono f) (i : Fin n) : f i = i := by
  exact DFunLike.congr_fun
    (OrderHom.eq_id_of_injective
      ((f, hf.monotone) : Fin n →o Fin n) hf.injective) i
```

The production patch fits the final application on one line and retains the original declaration header. The old declaration occupies 30 physical lines; the replacement occupies four. One direct import of `Mathlib.Data.Finset.Sort` makes the dependency explicit, leaving a net saving of 25 lines in the complete module.

Why the mathematics fits exactly

Injectivity on a finite set gives surjectivity. A finite chain has no nontrivial order automorphism: its least element must be fixed, then the least remaining element must be fixed, and so on. The source proof reconstructs this argument; the library theorem packages it. No assumption $n > 0$ is needed. For $n = 0$ there is no element to prove fixed, while the singleton case follows automatically.

The standalone file also checks the same statement for an arbitrary finite linear order. Linearity matters. A finite partial order with two incomparable elements admits their exchange as an order automorphism, so replacing the linear-order condition with a partial-order condition would be false.

Effect on the physical-order proof

The client `rankMap_eq` still obtains strict monotonicity from `rankMap_strictMono`. That theorem still depends on the actual descending weights and strict-rank facts. The replacement removes only the generic rigidity argument; it does not certify the numerical physical-order table or eliminate its coverage and ordering obligations.

The original and replacement standalone declarations both compiled, and both report precisely `propext`, `Classical.choice` and `Quot.sound`. An independent reviewer also compiled the replacement and checked the empty and singleton cases. No compilation-speed or compiled-object reduction is inferred from the shorter text.

Novelty boundary. Earlier proposals cover finite predecessor invariance, positive-tail physical-position injectivity, and construction of internal order from a strict relation. The audit found no proposal for this finite-chain endomorphism rigidity deletion in the newly added physical-order module. The contribution is this library reuse at the exact target, not a new order-theoretic theorem.

3 P2–P4: use the canonical residue APIs in the new controller proofs

The controller modules construct distinct finite subsets with prescribed affine residue sums. Their mathematical construction is already explicit and relatively concise. The additional opportunities lie in three proofs that manually reconstruct standard facts about canonical residue

representatives. Each patch retains the original theorem statement and the same constructed objects.

3.1 P2: bounded modular injectivity through congruences

Target. `ControllerIntervalSums.interval_injOn`.

For $a \le T$, the function $j \mapsto s + j$ is injective on the natural interval $0 \le j < a$, when its values are interpreted in $\mathbb{Z}/T\mathbb{Z}$. The old proof cancels s, applies `ZMod.val`, rewrites both natural casts as remainders, and simplifies the remainders using two local inequalities.

After cancellation, the exact intermediate statement is already $j \equiv k \pmod{T}$. Uniqueness of representatives below T is the standard `Nat.ModEq.eq_of_lt_of_lt` theorem:

```
intro j hj k hk he
exact ((ZMod.natCast_eq_natCast_iff j k T).mp
  (add_left_cancel he)).eq_of_lt_of_lt
  ((Finset.mem_range.mp hj).trans_le ha)
  ((Finset.mem_range.mp hk).trans_le ha)
```

The production formatting uses three tactic lines after the declaration, removing three physical lines from the module. This exposes the actual mathematical reason for injectivity rather than carrying out a second representative calculation.

The bound $a \le T$ remains essential. Without it, 0 and T can both lie in the interval and have equal casts. The proof supports an empty interval, a full interval $a = T$, arbitrary composite T, wraparound after translation, and $T = 1$. Independent checked examples include the empty interval, a full interval modulo six and modulus one.

3.2 P3: normalize the projection equality before taking its value

Target. `ControllerResidueLift.exists_coordinate`.

Suppose $z \in \mathbb{Z}/(C_0T)\mathbb{Z}$, $a < C_0$, and the projection of z to $\mathbb{Z}/C_0\mathbb{Z}$ equals the cast of a. The theorem constructs $u \in \mathbb{Z}/T\mathbb{Z}$ with

$$z = a + C_0 \,\mathrm{val}(u).$$

The witness is the natural quotient $\mathrm{val}(z)/C_0$, cast modulo T. The source already proves that this quotient is below T and reconstructs the equality with `Nat.mod_add_div`. Those parts need no alteration.

The avoidable part is the auxiliary homomorphism `phi` and the proof that `phiz` is the cast of `z.val`. The pinned `ZMod.castHom_apply` and `ZMod.cast_eq_val` lemmas already express that normal form. Apply `ZMod.val` to the original hypothesis directly:

```
have hm : z.val % C0 = a := by
  simpa only [ZMod.castHom_apply, ZMod.cast_eq_val,
    ZMod.val_natCast, Nat.mod_eq_of_lt ha]
    using congrArg ZMod.val hz
```

This removes five physical lines and one unnecessary locally named map. The important witness and quotient reasoning remain visible. The inequality $a < C_0$ is retained: the projected

residue determines a itself only when a is a canonical representative. Both nonzero-modulus assumptions remain. At $T = 1$, the quotient coordinate is zero; at $C_0 = 1$, the premise forces $a = 0$. Neither case needs a special proof branch.

This proposal does not change the earlier canonical-witness or Euclidean-division infrastructure. It removes one redundant projection proof in the new controller module.

3.3 P4: derive positivity directly from a nonzero projection

Target. `ControllerAuxiliaryChoice.exists_coprime_cardinality`.

After choosing a Boolean coefficient e, the source sets $r = t - eb$ and knows that its projection $\phi(r)$ to a prime modulus p is nonzero. It needs a cardinality $a = \mathrm{val}(r)$ with $0 < a < C_0$ and $\gcd(a, p^n) = 1$.

The existing proof first relates the cast of `r.val` to `phir`, then proves nondivisibility by contradiction, and finally derives positivity from another contradiction involving divisibility of zero. Two standard interfaces make the proof more direct:

```
have hnd : ¬ p | r.val := by
  apply (ZMod.natCast_eq_zero_iff r.val p).not.mp
  simpa only [phi, ZMod.castHom_apply, ZMod.cast_eq_val]
    using hr
have hpos : 0 < r.val :=
  ZMod.val_pos.mpr (fun hz => hr (by rw [hz, map_zero]))
```

The nondivisibility statement comes directly from the characteristic-p cast criterion. Positivity follows because a homomorphism sends zero to zero: $\phi(r) \neq 0$ implies $r \neq 0$, and the canonical value of a nonzero residue is positive. This proof keeps the dependency on primality at the final coprimality-with-p^n step, where it belongs.

The complete module loses five physical lines. Composite C_0 is allowed, and $n = 0$ remains supported. No assumption that C_0 is prime, or that n is positive, has been introduced. The two explicit numerical controls in the standalone coordinate proof also cover a nontrivial quotient and the zero-ring target $T = 1$.

Controller validation and cost

The three body edits remove 13 physical lines in total. Their public declarations and original imports are retained in the production patch. The supplied four-module validation overlay includes the unchanged `ControllerOrbitTransport` between the changed modules. It replaces only the broad `Mathlib.Tactic` import with a documented finite-sum import sufficient for the isolated chain. This lets the complete source bodies be compiled in the constrained environment; it does not establish that the original project's transitive import closure has been rebuilt.

Independent standalone checks cover all three replaced proofs. The coordinate-lift theorem reports `propext` and `Quot.sound`; the interval and coprime-cardinality proofs also use `Classical.choice`. The final controller replay passes nine invocations: four complete semantic modules, their four original focused check clients, and an additional boundary/axiom harness. The check overlay explicitly imports `Mathlib.Tactic.NormNum` and `Mathlib.Tactic.NormNum.Prime`. The packet

records these validation-import adaptations rather than presenting the result as an unqualified maintained-tree build.

4 P5: a nonzero integer certificate without a sign split

Target. The correctness field of `DiophantineConsistency.Rep.nonzero`.

After the unchanged expression-evaluation normalization, the goal is

$$z \neq 0 \quad \Longleftrightarrow \quad \exists w : \mathrm{Fin}(1) \to \mathbb{N}, \quad z^2 - 1 - w(0) = 0.$$

The representation itself already uses one natural witness. The old proof proves $1 \leq z^2$ by splitting $z \leq -1$ from $1 \leq z$, runs `nlinarith` on both branches, chooses the `toNat` of $z^2 - 1$, rewrites its cast, and uses `ring`.

The required fact is exactly square positivity plus integer discreteness:

$$z \neq 0 \iff 0 < z^2 \iff 1 \leq z^2 \iff 0 \leq z^2 - 1.$$

Use `mul_self_pos` and the integer successor-order equivalence, then let `Int.eq_ofNat_of_zero_le` supply the natural representative:

```
constructor
· intro hz
  obtain ⟨n, hn⟩ := Int.eq_ofNat_of_zero_le
    (Int.sub_nonneg.mpr
      (Int.add_one_le_iff.mpr (mul_self_pos.mpr hz)))
  exact ⟨fun _ => n, sub_eq_zero.mpr hn⟩
· rintro ⟨w, hw⟩
  apply mul_self_pos.mp
  have := Int.natCast_nonneg (w 0)
  omega
```

The remaining `omega` call handles a linear inequality with z^2 as a single atom. It does not need to prove a nonlinear square fact. Negative inputs and both units $z = \pm 1$ remain valid; zero still cannot satisfy the certificate because the natural slack is nonnegative.

The exact production formatting changes the 12-line arithmetic block to nine lines, for a net saving of **three physical lines**. This is a small maintenance improvement. No production helper or import is added. The witness count remains literally one, and the expression data remains byte-identical. The original and new exact arithmetic goals both compile and have the same three standard axioms. Their containing production module was not rebuilt.

The new contribution is the removal of sign analysis in this certificate. The equation, the one-witness representation, and general existential-witness normalization are not contributions of this packet.

5 P6: source positivity follows from selection, without marking

Target. `CurrentCarryConstruction2011.assembled_source_pos`.

The current theorem requires a stride s and a `MarkedWordNsa` premise. Its proof uses marking to show that the origin digit is one, then applies `word_pos_of_digit`. Positivity needs a positive digit somewhere; it does not need that digit to occur at the origin.

Every assembled cell already has its selected tile digit. The existing `word_selectedadummymarker0` theorem yields a digit equal to one at

$$i = \mathrm{base}(0) + \mathrm{physicalPosition}(a(0)).$$

Here the natural physical position is cast into `ZMod(N*2011)`. Consequently the stronger conclusion holds for every tile assignment and arbitrary natural control maps:

$$0 < \sum_{i \in \mathbb{Z}/(2011N)\mathbb{Z}} c(i)\,(2^{47})^{\mathrm{val}(i)}.$$

The selected term is positive and all other terms are natural numbers. The existing weighted-sum positivity lemma supplies the whole argument.

The contract that becomes simpler

The strengthened source theorem quantifies only over a, `dummy`, and `marker`, with the existing $N \neq 0$ condition. It drops the stride and marked-tiling premise. The exact replacement proof has two lines:

```
apply word_pos_of_digit _ _
exact (CurrentWordConstruction2011.word_selected
  a dummy marker 0).1.symm ▸ Nat.zero_lt_one
```

The production patch places the final expression on one line. The selected coordinate is inferred from the equality. This removes the proof's logical use of origin marking and horizontal/temporal matching. It does not remove those premises from conclusions that actually require matching or the exact origin digit.

The selector block lies below physical position 942, whereas the dummy slot is 977 and the marker slot is 1004. The existing selected-coordinate theorem has already discharged that separation. The new positivity proof consumes its exact equality and does not reproduce the disjointness argument.

For $N = 1$, cell zero still exists. If the selected tile is Start, the selected index may be zero and its contribution is $(2^{47})^0 = 1$; a nonzero position is unnecessary. Dummy and marker values may be larger than one or even larger than the radix, because such nonnegative values cannot invalidate positivity. They can invalidate Boolean normalization, so the strengthened positivity theorem is not a replacement for the Boolean hypotheses of normalization.

Source positivity is also distinct from positivity of the normalized transport representative. A positive product can be congruent to zero modulo the word modulus. The source theorem therefore makes no new claim about a positive normalized word, the high mask, or the remaining controller and polynomial endpoints.

Two explicit integration choices

The packet supplies mutually exclusive alternatives. The migration version strengthens the existing theorem in place and updates its only current check call. It changes the source-level

API by removing the final stride and marking arguments; external callers would need the same small argument deletion. The compatibility version introduces `assembled_source_pos_unmarked` and retains the old theorem as a one-line wrapper. That version grows the complete module from 108 to 118 physical lines, a cost of ten lines, and is included for users who need the old signature.

The migration changes the source module from 108 to 105 lines. Its 52-line check client changes only by deleting the two obsolete arguments at the sole application, retaining the old example statement and conclusion. Combined source and checks therefore change from 160 to 157 physical lines, and from 7,886 to 7,754 UTF-8 bytes. A maintained-source call-site search found no production client of the old theorem, only that existing check use and an axiom-print command. Thus there is no claimed present downstream source saving from a broad client migration. The benefit is the stronger theorem statement and the removal of an unnecessary premise at this specific boundary.

The independent Lean file proves the finite weighted-sum kernel and a source-shaped adapter whose only assembly input is the selected-coordinate equality. It checks single-cell examples with a selected origin and with selected index 941, allowing arbitrary natural controls. The actual production adapter and its full imports remain an integration check; the independent kernel is not described as a compiled production replacement.

6 Validation, integration and reproducibility

What the supplied evidence establishes

Proposal	Checked locally	Remaining integration scope
P1	Original/new exact theorem declarations, finite-order generalization, empty/singleton cases, axiom comparison	Production physical-order module and its downstream clients
P2–P4	Exact standalone bodies; four-module chain, four original checks and boundary harness with documented import adaptation; independent review	Original maintained import closure and ordinary repository build
P5	Original/new exact arithmetic goal, axiom comparison, expression-field preservation	Containing representation module and its clients
P6	Weighted-sum kernel, source-shaped selected-coordinate adapter and one-cell examples	Actual source-construction adapter and changed check client

All supplied production patches pass textual applicability checks at the initial pin. The closing refresh confirms that their target bytes are unchanged. Checked theorem declarations use only the standard axioms recorded in the receipts. The proposed Lean proofs introduce no `sorry`, new axiom, or `native_decide`. Existing imported proof infrastructure is not re-certified by this statement.

The controller overlay’s import replacement is a validation accommodation, not a proposed production-import optimization. Production before/after modules retain their original imports.

This distinction matters because replacing a broad imported module can affect what downstream legacy files see, even if the four local modules compile. Likewise, checking an isolated theorem body establishes its typed mathematical proof, but does not establish the complete project's build state.

The environment did not have Lean preinstalled and had very little ordinary temporary disk space. The pinned compiler and a selected mathlib cache were installed in temporary memory-backed space. This enabled the focused checks above, but a full production dependency build was not performed. No timing improvement, proof-object shrinkage, or complete downstream axiom comparison is claimed.

Suggested integration sequence

First apply P1 and run the physical-order module and its existing checks. Its exact contract is preserved and its source reduction is the largest. Next apply the three controller body changes as one small patch; compile the four modules in dependency order and their existing checks using the original imports. Apply P5 independently and build the arithmetic representation clients. Finally choose one P6 API alternative, depending on whether compatibility is needed, then build the construction module and its check client.

The bundle's `README.md` gives exact file names and commands. Its replay scripts run the standalone kernels and the isolated controller chain without applying any production patch. A separate verifier checks bundled hashes, before/after counts, target pins and patch applicability. The report's TeX source is self-contained and can be rebuilt with `latexmk -xelatex report.tex`.

7 Why several tempting directions were omitted

The exclusion audit materially narrowed the result. Generic schema existential elimination was rejected because prior packets already normalize determined witnesses under binders. A natural/integer rank cast shortcut was rejected because the exact rank bridge is already proposed. Generic reflected-parity/carry projection was also excluded: packet `o` explicitly identifies it as prior. The possible three-shift reduction of the fixed kernel was not promoted here because its incremental benefit over the assumed projection baseline was insufficiently established.

The report also omits scanner induction refactors, generic endpoint transport, common circuit proof generators, geometric-sum wrappers, shared internal-natural algebra, and fixed-index Pell anchors. Their omission is intentional even where maintained source still has older proofs. The adoption ledger changed during this review, illustrating why pending implementation status cannot be used as a novelty test.

Novelty here means that the audit found no matching concrete proposal in the requested corpus. It is not a claim of mathematical novelty or a guarantee against every possible paraphrase in historical material. The detailed exclusion records, archive hashes and candidate decisions make that judgment inspectable.

8 Further questions with concrete acceptance criteria

1. **Does P1 improve the maintained build beyond source length?** Measure the actual physical-order target after integration, with identical dependencies and options. Record elapsed time and object size separately. The standalone proof's brevity alone does not answer this.
2. **Can source positivity be consumed before tiling semantics?** Identify an actual construction endpoint that currently asks for marking solely to obtain positive source data. Replace only that use with P6, preserving Boolean and matching premises at their genuine consumers. No such production caller was found in the reviewed tree.
3. **Can the controller representative proofs remain local as the orbit interface grows?** Prefer the pinned library lemmas used in P2–P4 until repeated exact clients justify another helper. An added abstraction should be charged against its migrated clients and should retain composite-modulus and zero-exponent behavior.
4. **Do other nonzero certificates repeat sign analysis?** Search for the precise square-minus-one obligation after earlier normalization proposals have been applied. A new proposal should identify a different surviving obligation, rather than merely reusing P5 at an unnamed future site.

Artifact map

Location	Contents
`report.tex`, `report.pdf`	This self-contained report and its rendered form
`lean/`	Checked standalone kernels and original/replacement comparisons
`patches/`	Three independent proof-only patches and the two alternative positivity migrations
`overlays/`	Complete proposed source files, plus isolated controller validation sources
`evidence/`	Novelty inventory, corpus/ZIP hashes, source counts, refresh audit, compiler and independent-review receipts
`tools/`	Replay, integrity and patch-applicability commands

The archive is prepared for review and integration. No commit, pull request, or remote repository change was made during this work.